Лабораторна робота № 6

Тема роботи:

ПРОЄКТУВАННЯ БАЗИ ДАНИХ. РЕАЛІЗАЦІЯ СИСТЕМИ

Частина 1 ПРОЄКТУВАННЯ БАЗИ ДАНИХ

Мета роботи: створення схеми і генерація бази даних.

Завдання роботи: освоїти прийоми побудови схеми бази даних, генерації бази даних.

Зміст роботи:

1) створення компонента бази даних;
2) визначення стійких класів;
3) створення схеми бази даних;
4) генерація бази даних.

Приклад виконання лабораторної роботи

1. Проектування баз даних

Проектування реляційних баз даних виконується з використанням засобу Data Modeler. Його робота грунтується на відомому механізмі відображення об'єктної моделі в реляційну. Результатом є побудова діаграми «сутність-зв'язок» і подальша генерація опису БД на SQL.

Перед виконанням цієї лабораторної роботи треба виконати деякі налаштування. Натискаємо на меню Tools -> Extension Manager… . У вікні, що з'явилося, пишемо в пошук DDL (Data Definition Language), знаходимо цей додаток та клікаємо кнопку Install.

Цей застосунок виконує генерацію SQL запиту на основі діаграми сутність-зв'язок (Entity Relationship Diagram).

Результат будування діаграми сутність-зв'язок наведено на рис. 2.33.

Створення схеми БД:

1) створіть пакет (package) із назвою Schema;

2) в цьому пакеті створіть діаграму сутність-зв'язок (Add Diagram -> ER Diagram);

3) як результат з'явиться модель даних Data Model1, що матиме створену діаграму (рис 2.35).

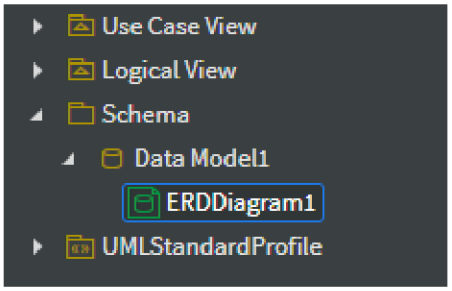

Рисунок 2.35 – Дерево проєкту на даному етапі

Щоб додати сутність:

1) відкрийте діаграму сутність-зв’язок;

2) оберіть з панелі інструментів елемент Entity;

3) клацніть по пустому місцю на діаграмі.

Щоб додати атрибут до сутності:

1) двічі клацніть по сутності лівою кнопкою миші;

2) оберіть Add Column (з правої сторони перший квадрат).

Щоб задати тип та інтервал атрибуту:

1) клацніть лівою кнопкою миші по атрибуту на діаграмі чи в Model Explorer;

2) знайдіть поле type і оберіть необхідний тип;

3) знайдіть поле length і введіть значення довжини.

Щоб додати зв’язок між сутностями:

1) оберіть на панелі інструментів один зі зв’язків: One to One Relationship, One to Many Relationship, Many to Many Relationship;

2) проведіть від однієї сутності до іншої.

Щоб визначити кратність кінців відношень:

1) натисніть лівою кнопкою миші по зв’язку;

2) знайдіть параметрів cardinality та оберіть значення для кожного кінця відношення (end1 та end2).

Щоб визначити Primary Key, Foreign Key, Unique:

1) оберіть потрібний атрибут на діаграмі;

2) поставте прапорець на потрібній властивості: primaryKey, foreignKey, unique.

Генерація опису БД на SQL:

1) перейдіть до меню Tools -> DDL -> Generate Code…;

2) у вікні, що відкрилося, оберіть для якої моделі даних генеруватиметься код (в нашому випадку DataModel1);

3) оберіть куди зберегти файл і дайте йому назву (формат файлу .txt).

Після завершення генерації відкрийте створений текстовий файл і перегляньте результати (рис. 2.37).

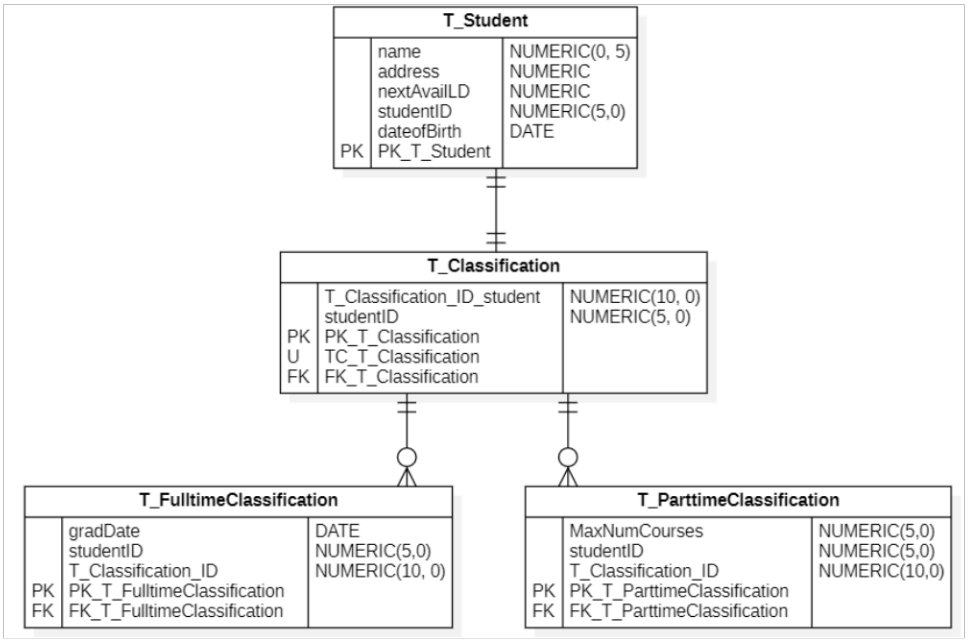

Рисунок 2.36 – Діаграма «сутність-зв’язок»

```sql
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS `T_Student`;
DROP TABLE IF EXISTS `T_Classification`;
DROP TABLE IF EXISTS `T_FulltimeClassification`;
DROP TABLE IF EXISTS `T_ParttimeClassification`;
SET FOREIGN_KEY_CHECKS = 1;

CREATE TABLE `T_Student` (
    `name` NUMERIC(0,5) NOT NULL,
    `address` NUMERIC NOT NULL,
    `nextAvailID` NUMERIC NOT NULL,
    `studentID` NUMERIC NOT NULL,
    `dateofBirth` DATE NOT NULL,
    `PK_T_Student` INTEGER NOT NULL,
    PRIMARY KEY (`PK_T_Student`)
);

CREATE TABLE `T_Classification` (
    `T_Classification_ID_student` NUMERIC(10,5) NOT NULL,
    `studentID` NUMERIC(5,0) NOT NULL,
    `PK_T_Classification` INTEGER NOT NULL,
    `TC_T_Classification` INTEGER NOT NULL,
    `FK_T_Classification` INTEGER NOT NULL,
    PRIMARY KEY (`PK_T_Classification`),
    UNIQUE (`TC_T_Classification`)
);

CREATE TABLE `T_FulltimeClassification` (
    `gradDate` NUMERIC(5,0) NOT NULL,
    `studentID` NUMERIC(5,0) NOT NULL,
    `T_Classification_ID` NUMERIC(10,0) NOT NULL,
    `PK_T_FulltimeClassification` INTEGER NOT NULL,
    `FK_T_FulltimeClassification` INTEGER NOT NULL,
    PRIMARY KEY (`PK_T_FulltimeClassification`)
);

CREATE TABLE `T_ParttimeClassification` (
    `maxNumCourses` NUMERIC(5,0) NOT NULL,
    `studentID` NUMERIC(5,0) NOT NULL,
    `T_Classification_ID` NUMERIC(5,0) NOT NULL,
    `PK_T_ParttimeClassification` INTEGER NOT NULL,
    `FK_T_Classification` INTEGER NOT NULL,
    PRIMARY KEY (`PK_T_ParttimeClassification`)
);
```

Рисунок 2.37 – SQL запит

Частина 2 РЕАЛІЗАЦІЯ СИСТЕМИ

Мета роботи: реалізація інформаційної системи.

Завдання роботи: освоїти прийоми реалізації компонентів системи, перевірки моделі на помилки, генерації коду системи.

Зміст роботи:

1) створення діаграми компонентів;
2) співвіднесення класів з компонентами;
3) генерація коду;
4) перевірка моделі.

Приклад виконання лабораторної роботи

1. Створення компонентів

Перед початком виконання роботи необхідно виконати певні налаштування. Натискаємо на меню Tools -> Extension Manager… . У вікні, що з'явилося, пишемо в пошук C++, знаходимо цей додаток та натискаємо кнопку Install.

У StarUML діаграми компонентів створюються в поданні компонентів системи. Окремі компоненти можна створювати безпосередньо на діаграмі або перетягувати їх туди з браузера.

Виберемо в якості мови програмування C++, і для класу Student створимо відповідні цій мові компоненти.

Створення діаграми компонентів (рисунок 2.38):

1) натискаємо правою кнопкою миші по пакету: Add Diagram -> Component Diagram;
2) двічі клацаємо по ній лівою кнопкою миші, аби відкрити.

Створення компонентів:

1) на панелі інструментів обираємо елемент Component;
2) клікаємо на пустому місці на діаграмі;
3) вводимо назву компоненту.

Створення специфікації компоненту:

1) обираємо компонент;
2) шукаємо поле stereotype;
3) вводимо значення Specification.

Щоб додати зв'язок залежності:

1) на панелі інструментів обираємо зв'язок Dependency;
2) натискаємо на залежний компонент та на компонент, від якого він залежить.

2. Генерація коду

Процес генерації коду складається з чотирьох основних кроків:

1) перевірка коректності моделі;
2) установка властивостей генерації коду;
3) вибір класу, компонента або пакета;
4) генерація коду.

Рисунок 2.38 – Діаграма компонентів

Якщо існують якісь помилки у моделі, то в нижньому правому куті буде відображатися знак попередження (рис. 2.39). Щоб переглянути помилки достатньо натиснути на цей знак.

Рисунок 2.39 – Зліва значок, який свідчить, що помилок немає, справа – помилки є

До найбільш поширених помилок відносяться такі, наприклад, як повідомлення на діаграмі після-послідовності або кооперативної діаграмі, не співвіднесені з операцією, або об’єкти цих діаграм, не співвіднесені з класом.

При генерації коду за один раз можна створити клас, компонент або цілий пакет. При генерації коду з пакету можна вибрати або пакет логічного представлення на діаграмі класів, або пакет представлення компонентів на діаграмі компонентів. При виборі пакета логічного представлення генеруються всі класи цього пакета. При виборі пакету представлення компонентів генеруються всі компоненти цього пакета.

Під час генерації коду StarUML вибирає інформацію з логічного і компонентного уявлень моделі і генерує великий обсяг «скелетного» (skeletal) коду:

1) класи – генеруються всі класи моделі;

2) атрибути – код включає атрибути кожного класу, в тому числі видимість, тип даних і значення за замовчуванням;

3) сигнатури операцій – код містить визначення операцій з усіма параметрами, типами даних параметрів і типом значення, що повертається операції;

4) зв’язку – деякі зі зв’язків моделі викликають створення атрибутів при генерації коду;

5) компоненти – кожен компонент реалізується у вигляді відповідного файлу з початковим кодом.

Для генерації коду C++:

1) перейдіть до меню Tools -> C++ -> Generate Code…;

2) оберіть пакет Logical View (код згенерується для всіх елементів, що знаходяться всередині цього пакету);

3) оберіть куди зберігати файли;

4) перегляньте згенерований код.

Фінальне дерево проєкту даної роботи виглядає як на рисунку 2.40.

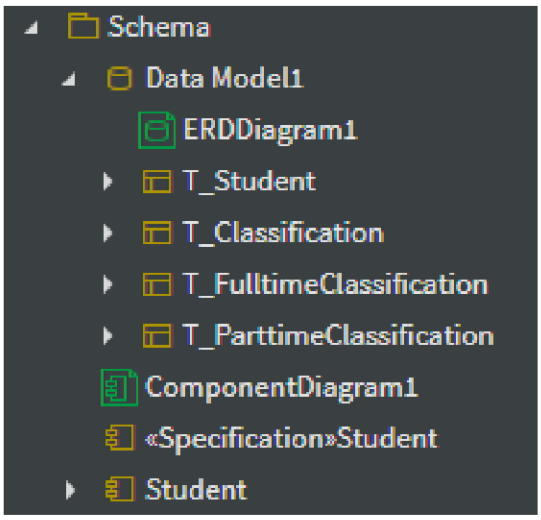

Рисунок 2.40 – Фінальне дерево проєкту

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

Основна

1. Басюк Т. М. Основи інформаційних технологій [Текст]: навч. посібн. / Т. М. Басюк, Н. О. Думанський, О. В. Пасічник [нове видання]. – Львів : «Новий Світ – 2000», 2017. – 390 с.

2. Шаховська Н. Б. Проектування інформаційних систем: Навчальний посібник / Н. Б. Шаховська, В. В. Литвин; за ред. В. В. Пасічника. – Львів: «Магнолія 2006», 2017. – 380 с.

3. Інформаційні системи і технології в економіці: Посібник/ За ред. В. С. Пономаренка. – К.: Академія, 2002. – 544 с.

4. Проектування інформаційних систем: Посібник / За ред. В. С. Пономаренка. – К.: Академія, 2002. – 488 с.

5. Гужва В. М. Інформаційні системи і технології на підприємствах: Навчальний посібник. – К.: КНЕУ, 2001. – 400 с.

Допоміжна

6. Авраменко В.С. Проектування інформаційних систем: навчальний посібник / В.С. Авраменко, А.С. Авраменко. – Черкаси: Черкаський національний університет ім. Б. Хмельницького, 2017. – 434 с.

7. Дудзяний І. М. Об’єктно-орієнтоване моделювання програмних систем: Навчальний посібник/ 5. Дудзяний І. М. – Львів: Видавничий центр ЛНУ імені Івана Франка, 2007. - 108 с.

8. Петрик М.Р. Моделювання програмного забезпечення : науково-методичний посібник / М.Р. Петрик, О.Ю. Петрик – Тернопіль : Вид-во ТНТУ імені Івана Пулюя, 2015. – 200 с.

9. Табунщик Г. В. Проектування та моделювання програмного забезпечення сучасних інформаційних систем / Г. В. Табунщик, Т.І. Каплієнко, О.А. Петрова − Запоріжжя : Дике Поле, 2016. – 250 с.

10. Томашевський О. М., Цегелик Г. Г., Вітер М. Б., Дудук В. І. Інформаційні технології та моделювання бізнес-процесів. Навч. посібник. / Телишевський О.М., Цегелик Г.Г., Вітер М.Б., Дідук В.І.– К.: Видавництво «Центр учбової літератури», 2012. – 296 с.